

-
1. `SELECT * FROM INFO;`
→ Retrieves all rows and columns from the INFO table.
 2. `SET SQL_SAFE_UPDATES=0;`
→ Turns off safe update mode to allow updates without key conditions (MySQL-specific).
 3. `ALTER TABLE INFO ADD COLUMN NON INT;`
→ Adds a new integer column named NON.
 4. `ALTER TABLE INFO DROP COLUMN NON;`
→ Deletes the column named NON.
 5. `UPDATE INFO SET CITY="DHAKA" WHERE NAME="GURUUU";`
→ Sets CITY to "DHAKA" for NAME "GURUUU".
 6. `SELECT NAME FROM INFO;`
→ Shows only the NAME column from INFO.
 7. `SELECT DISTINCT CITY FROM INFO;`
→ Returns unique CITY names.
 8. `SELECT NAME FROM INFO WHERE AGE>=23;`
→ Finds names with AGE 23 or above.
 9. `SELECT NAME FROM INFO WHERE AGE!=23;`
→ Finds names where AGE is not 23.
 10. `SELECT NAME FROM INFO WHERE CITY="DHAKA" or CITY="CUMILLA";`
→ valid
 11. `SELECT NAME FROM INFO WHERE CITY="DHAKA" OR CITY="CUMILLA";`
→ Finds names from DHAKA or CUMILLA.
 12. `SELECT NAME FROM INFO WHERE NOT AGE=23;`
→ Same as AGE != 23.
 13. `SELECT ID,NAME FROM INFO WHERE (CITY="DHAKA" OR CITY="CUMILLA") AND AGE=23;`
→ Gets ID and NAME of people aged 23 from DHAKA or CUMILLA.
 14. `SELECT * FROM INFO WHERE CITY IN("DHAKA", "CUMILLA");`
→ Same as using OR for multiple cities.
 15. `SELECT * FROM INFO WHERE AGE BETWEEN 20 AND 25;`
→ Gets records with AGE between 20 and 25 inclusive.
 16. `SELECT * FROM INFO WHERE NAME LIKE "M%";`
→ Names starting with M.
 17. `SELECT * FROM INFO WHERE NAME LIKE "%A%";`
→ Names containing A.
 18. `SELECT * FROM INFO WHERE NAME LIKE "%A";`
→ Names ending with A.
 19. `SELECT * FROM INFO WHERE NAME LIKE "A%A"`
→ Names starting and ending with A.
 20. `SELECT * FROM INFO WHERE NAME="NISHAT";`
→ Gets data of person named NISHAT.

21. `SELECT * FROM INFO WHERE NAME="ABIR";`
→ Gets data of person named ABIR.
 22. `SELECT MIN(AGE) FROM INFO;`
→ Returns the lowest age.
 23. `SELECT MAX(AGE) FROM INFO;`
→ Returns the highest age.
 24. `SELECT COUNT(AGE) FROM INFO;`
→ Counts how many non-null AGE values exist.
 25. `SELECT AVG(AGE) FROM INFO;`
→ Calculates average age.
 26. `SELECT SUM(AGE) FROM INFO;`
→ Adds all AGE values.
 27. `UPDATE INFO SET AGE=AGE+3 WHERE ID=101;`
→ Increases AGE by 3 for ID 101.
 28. `UPDATE INFO SET AGE=AGE-3 WHERE ID=101;`
→ Decreases AGE by 3 for ID 101.
 29. `SELECT SUM(AGE) AS TOTAL_AGE FROM INFO WHERE NAME IN ("ABIR", "NISHAT")
GROUP BY NAME;`
→ Sums age separately for ABIR and NISHAT.
 30. `SELECT SUM(AGE) AS TOTAL_AGE FROM INFO WHERE NAME IN ("ABIR",
"NISHAT");`
→ Sums total age of ABIR and NISHAT combined.
-
-

◆ 1 WHERE Clause

Purpose: Filters rows based on a condition.

Syntax:

```
SELECT column1, column2  
FROM table_name  
WHERE condition;
```

Example:

```
SELECT name, age  
FROM Students  
WHERE age > 18;
```

◆ 2 DISTINCT Keyword

Purpose: Returns only **unique** (non-duplicate) values.

Syntax:

```
SELECT DISTINCT column_name  
FROM table_name;
```

Example:

```
SELECT DISTINCT department  
FROM Employees;
```

◆ 3 Operators in WHERE Clause

Used to build logical conditions in the `WHERE` clause.

Operator	Description	Example
=	Equal	age = 20
<> or !=	Not Equal	age != 20
>	Greater than	salary > 5000
<	Less than	marks < 50
>=	Greater than or equal	age >= 18
<=	Less than or equal	age <= 60
AND	Both conditions true	age > 18 AND city = 'Dhaka'
OR	Any one condition true	dept = 'CSE' OR dept = 'EEE'
NOT	Negates the condition	NOT (city = 'Dhaka')

Example:

```
SELECT *  
FROM Employees  
WHERE salary > 5000 AND department = 'IT';
```



4 Aggregate Functions

Used to perform calculations on column values.

Function	Purpose
MIN()	Returns minimum

Function	Purpose
MAX ()	Returns maximum
AVG ()	Returns average
SUM ()	Returns total sum
COUNT ()	Returns row count

Syntax:

```
SELECT AGGREGATE_FUNCTION(column_name)
FROM table_name;
```

Examples:

```
-- Minimum Salary
SELECT MIN(salary) FROM Employees;

-- Maximum Salary
SELECT MAX(salary) FROM Employees;

-- Average Salary
SELECT AVG(salary) FROM Employees;

-- Total Salary
SELECT SUM(salary) FROM Employees;

-- Total Number of Employees
SELECT COUNT(*) FROM Employees;
```

Let me know if you want this as a **PDF, image card**, or to include **GROUP BY** and **HAVING** next!